Task 1:

```
.386
.model flat, stdcall
.stack 4096

.data

array dd 1,2,3,4,5
arySize dd 5
;Write a Procedure that takes 5 numbers as an argument. Calculate the sum of
;numbers and return the sum. Stores the sum in a variable in the main procedure.

.code
main proc

mov ecx, arySize
mov esi, offset array
L1:
        push [esi]
        add esi, 4
loop L1

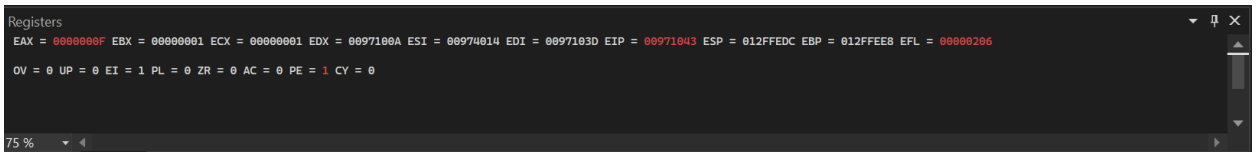
mov ecx, arySize
call sum

main endp

sum proc

        pop edi
        pop eax
        dec ecx
L2:
        pop ebx
        add eax,ebx
loop L2
        push edi
sum endp

end main
```



Task 2:

```
INCLUDE irvine32.inc
.386
.model flat, stdcall
.stack 4096

.data
        array   dd 5 DUP(?)
        arySize dd 5

        sum     dd 0
        avg     dd 0
        highest dd 0
        lowest  dd 0

        msg     byte "Enter elements: ", 0
        avgmsg   byte "Average: ", 0
        highestmsg byte "Highest: ", 0
        lowestmsg  byte "Lowest: ", 0
        summsg          byte "Sum: ", 0

.code
main PROC

        mov eax, 0
        mov ebx, 0
        mov esi, offset array
        mov ecx, arySize

inputLoop:
        mov edx, offset msg
        call writestring
        call readint
        mov [esi], eax
        add esi, 4
loop inputLoop

        call getTotal
        call getAverage
```

```
	call getHighest
	call getLowest

	; Display sum
	mov edx, offset summsg
	call WriteString
	mov eax, sum
	call WriteInt
	call CrLf

	; Display average
	mov edx, offset avgmsg
	call WriteString
	mov eax, avg
	call WriteInt
	call CrLf

	; Display highest
	mov edx, offset highestmsg
	call WriteString
	mov eax, highest
	call WriteInt
	call CrLf

	; Display lowest
	mov edx, offset lowestmsg
	call WriteString
	mov eax, lowest
	call WriteInt
	call CrLf

	exit
main ENDP

getTotal PROC

	mov ebx, 0
	mov eax, 0
	mov esi, offset array
	mov ecx, arySize
	totalLoop:
	mov ebx, [esi]
	add eax, ebx
	add esi,4
```

```
        loop totalLoop
        mov sum, eax
        ret
getTotal endp

getAverage PROC

        mov eax, sum
        mov ebx, arySize
        mov edx, 0
        div ebx
        mov avg, eax
        ret
getAverage endp

getHighest PROC
        mov esi, offset array
        mov ecx, arySize
        mov eax, [esi]
        add esi, 4

highestLoop:
        mov ebx, [esi]
        cmp ebx, eax
        jle skipUpdate
        mov eax, ebx

skipUpdate:
        add esi, 4
        loop highestLoop

        mov highest, eax
        ret
getHighest ENDP

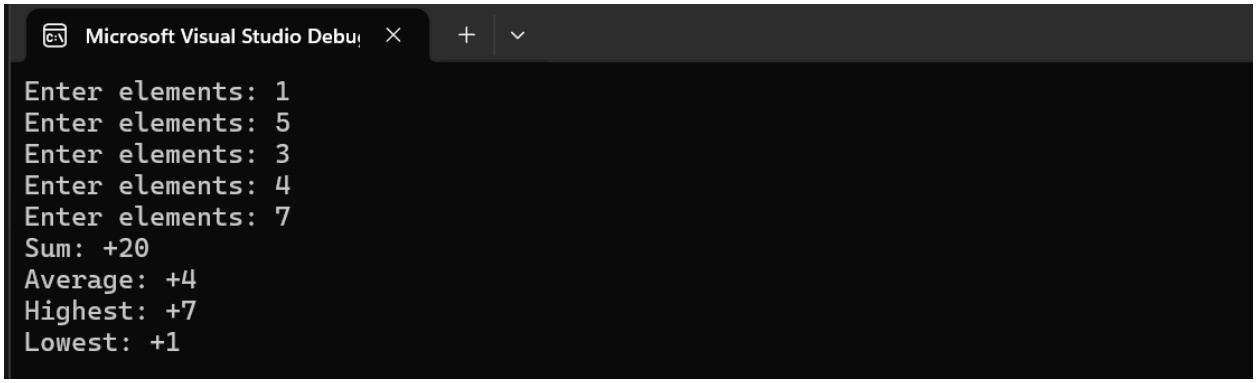
getLowest PROC
        mov esi, offset array
        mov ecx, arySize
        mov eax, [esi]
        add esi, 4

lowestLoop:
        mov ebx, [esi]
        cmp ebx, eax
```

```
        jge skipUpdate
        mov eax, ebx

skipUpdate:
        add esi, 4
        loop lowestLoop
        mov lowest, eax
        ret
getLowest endp


END main
```



Task 3:

```
INCLUDE irvine32.inc
.386
.model flat, stdcall
.stack 4096

.data
marks dd ?
gA byte "A",0
gB byte "B",0
gC byte "C",0
gD byte "D",0
msg byte "Enter Marks: ",0
inval byte "Invalid Input",0
.code
main PROC

        mov edx, offset msg
        call writestring
        call readint
```

```
        mov marks, eax
check1:
        cmp marks, 0
        JAE check2
        mov edx, offset inval
        jmp endMain
check2:
        cmp marks, 9
        JBE gradeD
        mov edx, offset inval
        jmp endMain
gradeD:
        cmp marks,5
        JAE gradeC
        mov edx, offset gD
        jmp endMain

gradeC:
        cmp marks,7
        JAE gradeB
        mov edx, offset gC
        jmp endMain

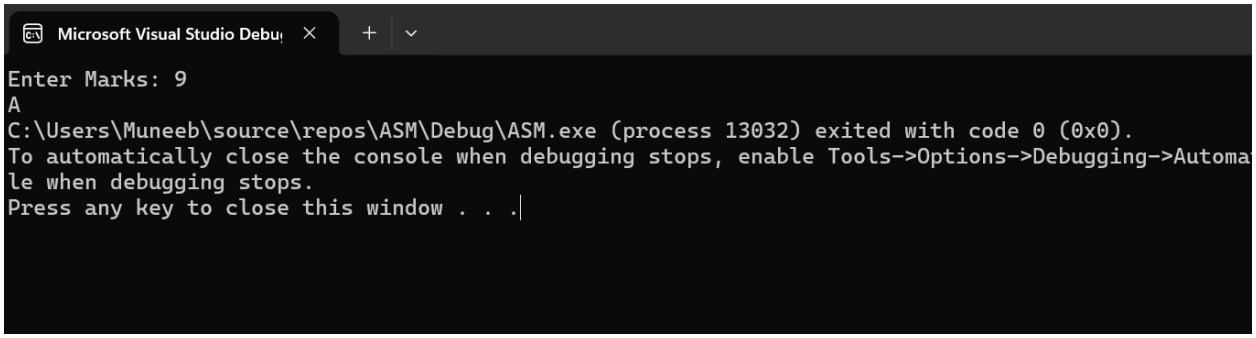
gradeB:
        cmp marks,9
        JAE gradeA
        mov edx, offset gB
        jmp endMain

gradeA:

        mov edx, offset gA
        jmp endMain



endMain:
call writestring
        exit
main ENDP
END main
```



Task 4:

```
INCLUDE irvine32.inc
.386
.model flat, stdcall
.stack 4096

.data
	userInput  dd ?
	result   dd ?
	prompt		byte "Enter a number: ", 0
	sqrtMsg		byte "Square root: ", 0

.code
main PROC

	mov edx, offset prompt
	call WriteString
	call ReadInt
	mov userInput, eax


	cmp eax, 0
	je returnResult
	cmp eax, 1
	je returnResult


	mov ebx, eax
	shr ebx, 1

sqrtLoop:
	mov eax, userInput
	mov edx, 0
	div ebx
```

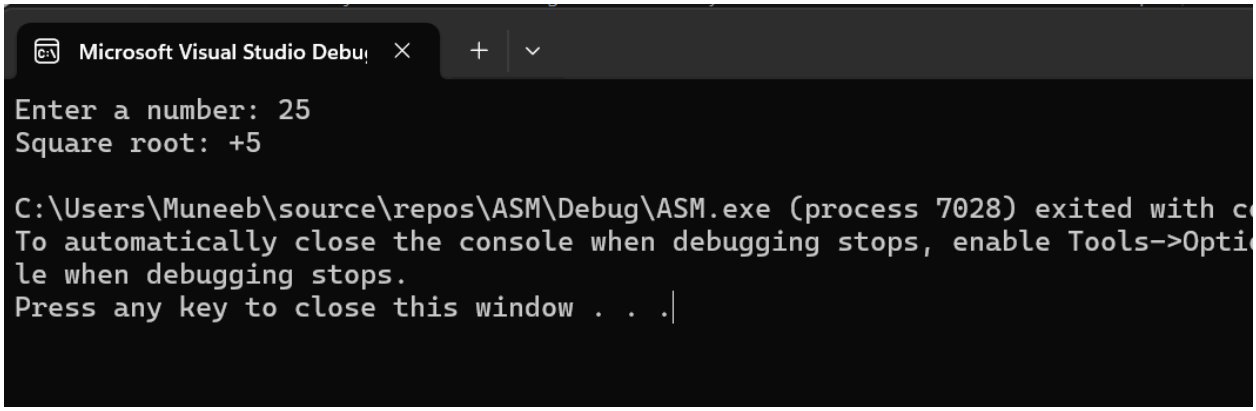
```
        add eax, ebx
        shr eax, 1
        cmp eax, ebx
        je done
        mov ebx, eax
        jmp sqrtLoop

done:
        mov result, eax

returnResult:

        mov edx, offset sqrtMsg
        call WriteString
        mov eax, result
        call WriteInt
        call CrLf

        exit
main ENDP
END main
```



Task 5:

```
INCLUDE irvine32.inc
.386
.model flat, stdcall
.stack 4096

.data
        enterString byte "Enter a string (max 32 characters): ",0
        enterNum1   byte "Enter first single-digit number: ",0
        enterNum2   byte "Enter second single-digit number: ",0
```

```
        menu            byte "Choose an option from the following:",0
        opt1    byte "1. Capitalize a string (32 Characters)",0
        opt2    byte "2. Multiply two single-digit numbers",0
        msg             byte 32 dup(?), 0
        strSize         dd 32
        var1    dd 0
        var2    dd 0
        result  dd 0
        choice          dd 0

.code
main PROC
        mov edx, offset menu
        call WriteString
        call CrLf

        mov edx, offset opt1
        call WriteString
        call CrLf

        mov edx, offset opt2
        call WriteString
        call CrLf

        call ReadInt
        mov choice, eax

        cmp choice, 1
        jne checkOp2
        call capitalize
        jmp endprog

checkOp2:
        cmp choice, 2
        jne endprog
        call mulNumbers
        jmp endprog

endprog:
        exit
main ENDP

capitalize PROC
        mov edx, offset enterString
```

```
        call WriteString

        mov edx, offset msg
        mov ecx, strSize
        call ReadString

        mov esi, offset msg

capitalizeLoop:
        mov al, [esi]
        cmp al, 0
        je endCapitalize

        cmp al, 'a'
        jb skipConvert
        cmp al, 'z'
        ja skipConvert

        sub al, 32
        mov [esi], al

skipConvert:
        inc esi
        jmp capitalizeLoop

endCapitalize:
        mov edx, offset msg
        call WriteString
        call CrLf

        ret
capitalize endp

mulNumbers PROC
        mov edx, offset enterNum1
        call WriteString
        call ReadInt
        mov var1, eax

        mov edx, offset enterNum2
        call WriteString
        call ReadInt
        mov var2, eax
```

```
        mov eax, 0
        mov ecx, var2

multiplyLoop:
        add eax, var1
        loop multiplyLoop   ;mul not working for some reason

        mov result, eax

        mov edx, offset result
        call WriteInt
        call CrLf

        ret
mulNumbers endp

END main
```

